

MEDLINK

Unified Digital Healthcare Platform

Graduation Project Documentation

Project Type: Digital Health / Healthcare Technology Platform

Target Market: Egypt → MENA → Global Expansion

Architecture: Scalable Modular Architecture / Future Microservices

Frontend: Angular

Backend: Node.js / Express.js

Database: MongoDB

Real-Time Communication: WebSocket / Socket.IO

Authentication: JWT + Refresh Tokens

Medical Data Standard: FHIR-Compatible Data Model

Project Team: 6 Members

1. Executive Summary

Medlink is a unified digital healthcare platform designed to solve one of the major problems in healthcare: fragmented patient information and unorganized communication between patients and healthcare providers.

Today, a patient may visit several doctors, laboratories, radiology centers, pharmacies, and hospitals. Each organization may have its own records, while the patient carries medical reports, prescriptions, scans, and test results manually.

At the same time, doctors often communicate with patients through personal phone numbers, WhatsApp, paper prescriptions, and disconnected systems.

MediVault aims to create a centralized digital healthcare ecosystem where the patient owns a unified digital medical profile and can securely share selected medical information with authorized healthcare providers.

The platform combines:

- Doctor discovery
- Appointment booking
- Digital medical records
- Medical history timeline

- Digital prescriptions
- Laboratory and radiology records
- Patient-doctor communication
- Follow-up management
- Medication reminders
- Consent and access management
- Notifications
- AI-assisted healthcare features
- Future integration with pharmacies, hospitals, laboratories, insurance companies, and other healthcare systems

The long-term vision is to transform MediVault from an appointment-booking platform into a Digital Healthcare Infrastructure Platform.

2. Problem Statement

2.1 Current Healthcare Problems

Patients face several problems when dealing with healthcare providers.

Problem 1 — Fragmented Medical History

A patient's medical information can be distributed across:

- Paper prescriptions
- Laboratory reports
- Radiology reports
- Hospital records
- Different doctors
- Mobile phone photos
- WhatsApp conversations
- Emails
- Physical folders

This makes it difficult to create a complete medical history.

Problem 2 — Repeating Medical History

When a patient visits a new doctor, they may need to explain:

- Previous diseases
- Previous surgeries
- Allergies
- Current medications
- Previous diagnoses
- Previous laboratory results
- Previous radiology results

This consumes time and can lead to missing information.

Problem 3 — Unorganized Doctor Communication

Patients commonly use:

- Phone calls
- WhatsApp
- Personal messages

for follow-up questions.

This creates problems for doctors because:

- Communication is not organized.
 - Patient information is distributed across different channels.
 - Doctors may receive messages outside working hours.
 - There is no structured consultation history.
 - Personal phone numbers may be exposed.
-

Problem 4 — Paper-Based Prescriptions

Traditional prescriptions can be:

- Lost
- Damaged
- Misread
- Difficult to track

Patients may also forget:

- Medication timing
- Dosage

- Duration
 - Refills
-

Problem 5 — Lack of Data Ownership and Consent

Patients often have limited visibility into:

- Who accessed their medical information
- What information was accessed
- When it was accessed
- How long access remains active

MediVault addresses this through explicit consent and access control.

3. Proposed Solution

MediVault provides a centralized digital healthcare platform.

The patient creates a secure digital medical profile.

The patient can:

1. Book appointments.
2. Store medical documents.
3. Organize medical history.
4. Receive digital prescriptions.
5. Track medications.
6. Communicate with doctors.
7. Share medical information temporarily.
8. Control access permissions.
9. Receive healthcare reminders.
10. Use AI-assisted features.

Doctors can:

1. Manage their profiles.
2. Manage appointments.
3. Access authorized patient information.
4. Create medical notes.
5. Create prescriptions.

6. Communicate with patients.
 7. Manage follow-ups.
 8. Review patient history.
 9. Track previous visits.
 10. Use AI-assisted tools.
-

4. Project Vision

Vision

To build a trusted digital healthcare ecosystem where medical information can securely follow the patient instead of remaining trapped inside individual hospitals, clinics, laboratories, or paper documents.

Vision Statement

"One patient. One secure health profile. Connected healthcare everywhere."

5. Project Mission

MediVault aims to:

- Improve healthcare continuity.
 - Reduce medical information fragmentation.
 - Improve doctor-patient communication.
 - Reduce dependency on paper records.
 - Give patients greater control over their health data.
 - Help doctors access relevant information faster.
 - Build an infrastructure capable of integrating healthcare providers.
-

6. Target Users

The platform contains several user types.

6.1 Patient

The main consumer of the platform.

Capabilities:

- Account management
 - Medical profile
 - Medical history
 - Documents
 - Appointments
 - Prescriptions
 - Medication reminders
 - Doctor communication
 - Consent management
 - Notifications
-

6.2 Doctor

Capabilities:

- Professional profile
 - Appointment management
 - Patient management
 - Medical notes
 - Prescriptions
 - Follow-up
 - Patient communication
 - Medical history access
-

6.3 Clinic / Medical Center

Capabilities:

- Multiple doctors
- Receptionist accounts
- Appointment management
- Patient management
- Doctor schedules
- Reports
- Staff management

6.4 Laboratory

Future integration.

Capabilities:

- Receive laboratory requests
- Upload results
- Link results to patient profile
- Digital verification

6.5 Radiology Center

Capabilities:

- Upload radiology reports
- Upload medical images
- Connect results to patient record

6.6 Pharmacy

Future module.

Capabilities:

- Receive electronic prescriptions
- Verify prescription
- Prepare medications
- Delivery integration

6.7 Hospital

Future enterprise integration.

Capabilities:

- Patient admission
 - Emergency access
 - Medical history
 - Clinical records
 - Discharge summaries
-

6.8 Insurance Company

Future integration.

Capabilities:

- Insurance verification
 - Coverage validation
 - Pre-authorization
 - Claims integration
-

6.9 System Administrator

Capabilities:

- User management
 - Provider verification
 - System monitoring
 - Reports
 - Security monitoring
 - Complaints
 - Audit logs
-

7. Core Value Proposition

For Patients

MediVault provides:

"Your entire healthcare journey in one secure place."

Patients no longer need to depend completely on:

- Paper
 - WhatsApp
 - Phone calls
 - Scattered files
-

For Doctors

MediVault provides:

"A structured digital workspace for patient care."

Doctors can manage:

- Patients
- Appointments
- Medical notes
- Prescriptions
- Follow-ups
- Communication

from one platform.

For Healthcare Providers

MediVault provides:

"A digital infrastructure for connecting healthcare services."

8. Main System Modules

The platform consists of the following major modules:

1. Authentication & Identity
2. Patient Management
3. Doctor Management

4. Healthcare Provider Management
 5. Appointment Management
 6. Medical Records
 7. Medical Timeline
 8. Document Management
 9. Prescription Management
 10. Medication Management
 11. Patient-Doctor Chat
 12. Follow-up Management
 13. Consent Management
 14. Notifications
 15. Search & Discovery
 16. Reviews & Ratings
 17. AI Services
 18. Laboratory Integration
 19. Pharmacy Integration
 20. Insurance Integration
 21. Administration
 22. Analytics
 23. Audit & Security
-

9. Authentication & Identity

The authentication system provides secure access to the platform.

Features

- Registration
 - Login
 - Logout
 - Email verification
 - Phone verification
 - Password reset
 - Refresh tokens
 - Session management
 - Device management
 - Account locking
 - Two-factor authentication
 - Role-based authorization
-

10. User Roles

Example roles:

PATIENT
DOCTOR
RECEPTIONIST
CLINIC_ADMIN
LAB_ADMIN
RADIOLOGY_ADMIN
PHARMACY_ADMIN
HOSPITAL_ADMIN
INSURANCE_ADMIN
SYSTEM_ADMIN

A user can potentially have multiple roles depending on the architecture.

11. Patient Profile

The patient profile contains:

Basic Information

- Full name
- Date of birth
- Gender
- Phone
- Email
- Emergency contact
- Profile picture

Medical Information

- Blood type
- Allergies
- Chronic conditions
- Previous surgeries
- Current medications
- Family medical history

Sensitive medical fields should have additional security controls.

12. Universal Health ID

Each patient receives a unique:

Universal Health ID

Example:

MV-2026-9A73K21

The ID is not intended to replace government identification.

It acts as a platform-level medical identity.

The patient can also have:

QR Code

associated with the health profile.

13. Medical Record

The medical record is the core of the platform.

It contains:

Patient

- Personal Information
 - Allergies
 - Conditions
 - Medications
 - Surgeries
 - Visits
 - Diagnoses
 - Prescriptions
 - Lab Results
 - Radiology
 - Documents
 - Vaccinations
-

14. Medical Timeline

Instead of presenting medical records as disconnected documents, MediVault creates a chronological timeline.

Example:

2026-08-20

Doctor Visit

↓

Diagnosis

↓

Prescription

↓

Lab Request

2026-08-22

Laboratory Result

2026-08-25

Follow-up

2026-09-01

New Prescription

This allows doctors and patients to understand the patient's healthcare journey.

15. Medical Documents

Patients can upload:

- Blood tests
- X-rays
- CT scans
- MRI reports
- Prescriptions
- Medical reports
- Discharge summaries
- Vaccination records
- Other medical documents

Each document contains metadata:

Document ID

Patient ID
Document Type
Upload Date
Document Date
Provider
Description
File URL
Verification Status

16. Smart OCR

MediVault can optionally use OCR to extract information from uploaded documents.

Example:

The patient uploads:

CBC Blood Test

The OCR service extracts:

Hemoglobin: 13.4
WBC: 7.2
Platelets: 250

The system can store the structured values while preserving the original document.

Important:

AI/OCR output should be treated as extracted information and not as an independent medical diagnosis.

17. Lab Results Tracking

The system can transform laboratory results into historical data.

Example:

Blood Glucose

Jan → 110
Mar → 125
Jun → 118

Aug → 130

The patient can view trends.

Doctors can see historical measurements.

18. Appointment System

Patients can:

- Search doctors
- Filter by specialty
- View doctor profiles
- View available schedules
- Book appointments
- Reschedule
- Cancel
- Receive reminders

Doctors can:

- Define working hours
 - Define appointment duration
 - Block unavailable times
 - Approve or reject appointments
 - Manage appointment status
-

19. Appointment Lifecycle

Example:

```
PENDING
↓
CONFIRMED
↓
CHECKED_IN
↓
IN_PROGRESS
↓
COMPLETED
```

Alternative states:

CANCELLED
NO_SHOW
RESCHEDULED

20. Doctor Profile

Doctor profile contains:

- Name
 - Specialty
 - Sub-specialty
 - Medical degree
 - Experience
 - Clinic locations
 - Consultation fee
 - Available appointments
 - Languages
 - Professional bio
 - Ratings
 - Verification status
-

21. Doctor Verification

Healthcare providers should not automatically receive verified status.

The platform can implement:

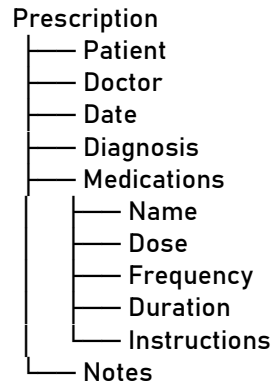
Pending Verification
↓
Documents Submitted
↓
Admin Review
↓
Approved / Rejected

Possible verification documents include appropriate professional credentials.

22. Digital Prescription

Doctors can create electronic prescriptions.

Prescription structure:



The patient receives the prescription digitally.

23. Medication Reminder

Patients can configure reminders.

Example:

Medicine: Medication A
Dose: 500mg
Frequency: 2 times/day
Time:
08:00 AM
08:00 PM
Duration:
7 days

Notifications remind the patient.

24. Follow-Up System

Doctors can create follow-up plans.

Example:

Follow-up Required

Date: 2026-09-15

Reason: Blood pressure review

The patient receives a reminder.

25. Patient-Doctor Communication

Instead of exposing personal phone numbers, MediVault provides an internal communication system.

Features:

- Real-time messaging
 - Text messages
 - Attachments
 - Medical document sharing
 - Read status
 - Notifications
 - Conversation history
-

26. Controlled Communication

Chat should not become an unlimited replacement for medical care.

Possible model:

Completed Appointment
↓
Follow-up Chat Activated
↓
Valid for 7 Days
↓
Chat Closed

Or:

Doctor-defined communication window

This protects doctors from unlimited communication.

27. Paid Follow-Up

Future monetization feature.

Doctors can offer:

Follow-up Consultation
Price: X EGP
Duration: 7 Days

The patient pays through an integrated payment provider.

28. Consent Management

Consent is one of the most important features.

Patients control access to their medical information.

Example:

Doctor A
Access:
Medical History
Prescriptions
Lab Results

Duration:
24 Hours

After expiration:

ACCESS REVOKED

29. Temporary Medical Access

The patient can generate:

QR Code

The doctor scans it.

The patient confirms access.

The doctor receives access to selected information.

Example:

Access Scope:

- ✓ Allergies
 - ✓ Medications
 - ✓ Previous Diagnoses
 - ✓ Lab Results
 - ✗ Private Documents
 - ✗ Other Doctors' Notes
-

30. Emergency Access

Future feature.

In emergencies, authorized emergency personnel may request limited access.

The system should record:

Who accessed
When
Why
What was accessed

Emergency access must have strict policy and audit requirements.

31. Audit Logs

Every sensitive action should be logged.

Example:

Doctor Ahmed accessed Patient X medical record.

Date:

2026-09-04

Time:
18:35

Action:
VIEW_MEDICAL_RECORD

IP:
...

Device:
...

Patients can potentially view their access history.

32. Notifications

Notification types:

Appointment

- Appointment confirmed
- Appointment reminder
- Appointment cancelled
- Doctor running late

Medical

- New prescription
- New lab result
- Follow-up reminder

Security

- New login
- Medical record accessed
- New consent request

Communication

- New message
- New consultation request

33. AI Layer

AI should be treated as an assistant rather than a replacement for doctors.

The platform can include several AI services.

34. AI Patient Assistant

The assistant can help patients navigate the platform.

Examples:

"Show my latest blood test."

"When is my next appointment?"

"What medications did my doctor prescribe?"

"Help me find a cardiologist."

For symptom-related interactions, the system should provide appropriate safety-oriented guidance and escalation rather than presenting an AI-generated diagnosis as fact.

35. AI Pre-Assessment

Before an appointment, the patient can answer structured questions.

Example:

Main Complaint:
Chest discomfort

Duration:
2 days

Severity:
5/10

Associated symptoms:

...

The system generates a structured summary for the doctor.

36. Medical Voice-to-Text

Doctors can dictate notes.

Example:

Doctor says:

Patient reports headache for three days...

System generates:

Clinical Note:

Patient reports headache for three days...

The doctor reviews and confirms the generated content.

37. AI Medical Summary

For patients with extensive medical records, the system can generate a structured summary:

Medical Summary

Chronic Conditions:

...

Allergies:

...

Current Medications:

...

Recent Visits:

...

Recent Lab Results:

...

The doctor remains responsible for clinical interpretation.

38. AI Drug Interaction Alerts

Future feature.

The system can compare:

Current Medication
+
Previous Medications
+
Known Allergies

and generate alerts for potential issues.

Any clinical decision-support functionality must be validated carefully and should not be treated as a substitute for professional judgment.

39. AI Architecture

Possible architecture:

```
graph TD; A[Angular Application] --> B[Node.js API]; B --> C[AI Service]; C --> D[LLM / ML Model]; D --> E[Structured Medical Context]; E --> F[Response];
```

AI services should not have unrestricted access to the entire database.

Only the minimum required data should be provided.

40. Search & Discovery

Patients can search for:

- Doctors
- Specialties
- Clinics
- Hospitals
- Laboratories
- Radiology centers

Filters:

Specialty
Location
Price
Rating
Availability
Gender
Language
Experience

41. Reviews and Ratings

Patients can rate doctors after completed appointments.

Example:

Overall Rating
Communication
Waiting Time
Professionalism
Clinic Experience

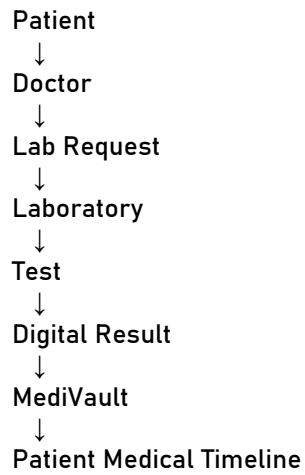
Anti-abuse mechanisms should be implemented to reduce fake reviews.

42. Laboratory Integration

Future version.

Laboratories can integrate with MediVault using APIs.

Flow:



43. Radiology Integration

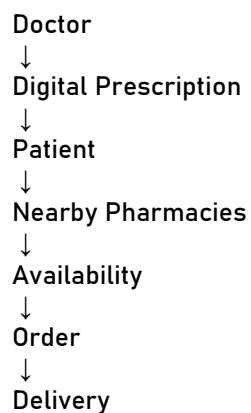
Radiology centers can upload:

- Reports
- Images
- Metadata

The system can associate them with the patient's medical record.

44. Pharmacy Integration

Future workflow:



This transforms the platform from a healthcare information system into a healthcare services ecosystem.

45. Hospital Integration

Hospitals can connect using standardized APIs.

Potential information:

- Admissions
- Discharge summaries
- Diagnoses
- Procedures
- Lab results
- Medications

The patient remains the owner/controller of access according to applicable policies and permissions.

46. Insurance Integration

Future integration:

Doctor
↓
Medical Service
↓
Insurance Verification
↓
Coverage Check
↓
Pre-Authorization
↓
Service

This can reduce administrative delays.

47. FHIR Compatibility

The platform should design its medical data model with interoperability in mind.

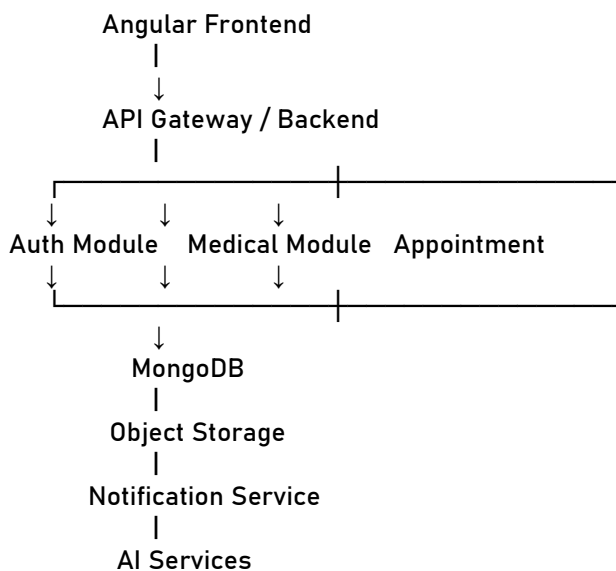
Potential FHIR resources include:

Patient
Practitioner
Organization
Encounter
Observation
Condition
Medication
MedicationRequest
DiagnosticReport
DocumentReference
Appointment
AllergyIntolerance

FHIR compatibility makes future integration easier with external healthcare systems.

48. High-Level Architecture

Initial architecture:



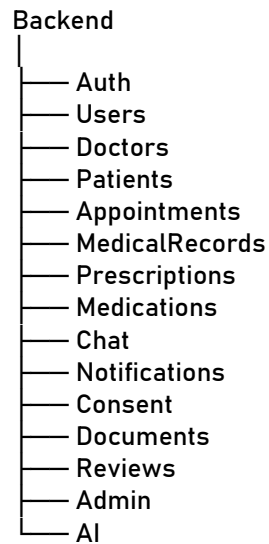
49. Recommended Architecture for Graduation Project

For the first version, use a:

Modular Monolith

instead of immediately implementing full Microservices.

Structure:



This is much easier for a 6-person graduation team.

The modules can later be extracted into Microservices.

50. Why Modular Monolith?

Because Microservices introduce additional complexity:

- Service discovery
- Network communication
- Distributed transactions
- Monitoring
- Deployment
- Multiple databases
- Message brokers
- Infrastructure management

For a graduation project, complexity should serve the product rather than become the product.

The system should therefore be designed so that future migration to Microservices is possible.

51. Backend Technology

Recommended stack:

Node.js
Express.js
TypeScript
MongoDB
Mongoose
Socket.IO
Redis
JWT
Cloud/Object Storage

Optional:

RabbitMQ / Kafka
Docker
Nginx
CI/CD

52. Frontend Technology

Recommended:

Angular
TypeScript
RxJS
Angular Router
Reactive Forms
HTTP Client
Signals
Guards
Interceptors

UI:

Angular Material

or
Tailwind CSS

53. Database

MongoDB is suitable for the project because healthcare data contains multiple document-oriented entities.

Possible collections:

users
patients
doctors
organizations
appointments
medicalRecords
encounters
diagnoses
conditions
prescriptions
medications
labResults
documents
consents
chatRooms
messages
notifications
reviews
auditLogs

54. Example User Document

```
{  
  "_id": "...",  
  "role": "PATIENT",  
  "email": "patient@example.com",  
  "phone": "...",  
  "passwordHash": "...",  
  "isVerified": true,  
  "createdAt": "..."  
}
```

Sensitive information should not be stored unnecessarily in the authentication document.

55. Example Patient Document

```
{
  "_id": "...",
  "userId": "...",
  "healthId": "MV-2026-9A73K21",
  "dateOfBirth": "...",
  "bloodType": "...",
  "allergies": [],
  "conditions": [],
  "emergencyContact": {},
  "createdAt": "...",
  "updatedAt": "..."
}
```

56. Appointment Data Model

Appointment

id
patientId
doctorId
organizationId
date
startTime
endTime
type
status
reason
notes
createdAt
updatedAt

57. Prescription Data Model

Prescription

id
patientId
doctorId
appointmentId
diagnosis
medications[]
notes
issuedAt
expiresAt

status

Medication:

name
dose
frequency
route
duration
instructions

58. Consent Data Model

Consent

id
patientId
granteeId
granteeType
permissions[]
startDate
expirationDate
status
createdAt
revokedAt

Example permissions:

VIEW_PROFILE
VIEW_MEDICAL_HISTORY
VIEW_LABS
VIEW_PRESCRIPTIONS
VIEW_DOCUMENTS
VIEW_MEDICATIONS

59. Chat Data Model

ChatRoom

id
patientId
doctorId
appointmentId
status
expiresAt
createdAt

Message:

```
id
roomId
senderId
messageType
content
attachments[]
createdAt
readAt
```

60. REST API Structure

Example:

```
/api/v1/auth
/api/v1/users
/api/v1/patients
/api/v1/doctors
/api/v1/appointments
/api/v1/medical-records
/api/v1/prescriptions
/api/v1/medications
/api/v1/documents
/api/v1/consents
/api/v1/chat
/api/v1/notifications
/api/v1/reviews
/api/v1/admin
```

61. Authentication APIs

```
POST /auth/register
POST /auth/login
POST /auth/refresh
POST /auth/logout
POST /auth/verify-email
POST /auth/forgot-password
POST /auth/reset-password
```

62. Appointment APIs

```
GET /appointments
GET /appointments/:id
```

POST /appointments
PATCH /appointments/:id
DELETE /appointments/:id
POST /appointments/:id/cancel
POST /appointments/:id/reschedule

63. Medical Record APIs

GET /patients/:id/medical-record
GET /patients/:id/timeline
GET /patients/:id/conditions
GET /patients/:id/medications
GET /patients/:id/labs
GET /patients/:id/documents

Access must be authorized.

A doctor should not simply access a patient by changing an ID in the URL.

64. Prescription APIs

POST /prescriptions
GET /prescriptions/:id
GET /patients/:id/prescriptions
PATCH /prescriptions/:id

65. Consent APIs

POST /consents
GET /consents
GET /consents/:id
POST /consents/:id/revoke
POST /consents/:id/approve

66. Real-Time Communication

Socket.IO can handle:

New message

Typing indicator
Read status
Online status
Notifications
Appointment updates

Example:

Patient
↓
Socket
↓
Chat Server
↓
Doctor

67. File Storage

Medical documents should not be stored directly inside MongoDB as large binary objects for the main design.

Instead:

MongoDB
↓
Metadata
↓
Object Storage
↓
Encrypted File

Possible storage:

AWS S3
Azure Blob Storage
Google Cloud Storage

For the graduation environment, local object storage such as MinIO can also be considered.

68. Security Architecture

Security is a primary requirement.

The platform should implement:

- HTTPS
- Password hashing
- JWT authentication
- Refresh token rotation
- Role-Based Access Control
- Resource-level authorization
- Encryption at rest
- Encryption in transit
- Secure file access
- Audit logs
- Rate limiting
- Input validation
- Sanitization
- Security headers
- Session/device management
- Backup strategy

69. Authentication Security

Passwords should never be stored directly.

Use:

Argon2
or
bcrypt

Recommended authentication:

Access Token
+
Refresh Token

Access tokens should be short-lived.

70. Authorization

Authentication answers:

Who are you?

Authorization answers:

Are you allowed to access this resource?

Example:

A doctor may have:

VIEW_PATIENT_RECORD

but only when:

Patient Consent = ACTIVE

or another legally/policy-authorized basis exists.

71. Encryption

Sensitive data should be protected during transmission:

Client
↓ HTTPS/TLS
Backend

Sensitive stored data should also have appropriate encryption controls.

72. Privacy

Healthcare data is highly sensitive.

The system should follow applicable Egyptian data-protection and healthcare requirements.

Internationally, standards such as HIPAA may be useful as security/privacy design references, but HIPAA should not be described as the Egyptian legal requirement.

For a real commercial deployment, legal and compliance review is mandatory.

73. Data Ownership

The platform should be designed around the principle:

Patient data must not be freely accessible to every healthcare provider.

Access should be:

Requested
↓
Authorized
↓
Scoped
↓
Time Limited
↓
Audited

74. Non-Functional Requirements

Performance

Target:

API response:
< 500ms

for typical operations under normal load.

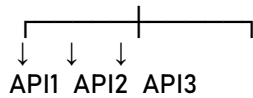
Critical operations should be optimized independently.

Scalability

The system should support horizontal scaling.

Example:

Load Balancer
|



Availability

The production target should be high availability.

Example target:

99.9%+

depending on infrastructure and deployment.

Reliability

The system must prevent:

- Data loss
 - Duplicate appointments
 - Duplicate prescriptions
 - Unauthorized access
-

75. Appointment Concurrency

One important backend problem is:

Two patients trying to book the same appointment simultaneously.

Solution:

Availability Check

+

Database Constraint / Atomic Operation

+

Transaction where required

The server must be the final authority.

The frontend must never be trusted to reserve a slot.

76. Caching

Redis can be used for:

- Doctor availability
- Sessions
- Rate limiting
- Temporary data
- Notification queues

Do not cache highly sensitive medical information casually.

77. Background Jobs

Some operations should run asynchronously.

Examples:

Send email
Send notification
OCR document
Generate AI summary
Process uploaded file
Create reminder

Architecture:

API
↓
Queue
↓
Worker
↓
Processing

78. Monitoring

Production system should monitor:

CPU
Memory
API latency
Error rate
Database performance
Queue length
Authentication failures
Security events

Tools can include:

Prometheus
Grafana
Sentry
ELK/OpenSearch

79. Logging

Logs should contain useful technical information without unnecessarily exposing medical content.

Example:

REQUEST_ID
USER_ID
ACTION
TIMESTAMP
STATUS
LATENCY

Medical content should not be casually written into application logs.

80. Backup and Disaster Recovery

Medical information requires strong backup planning.

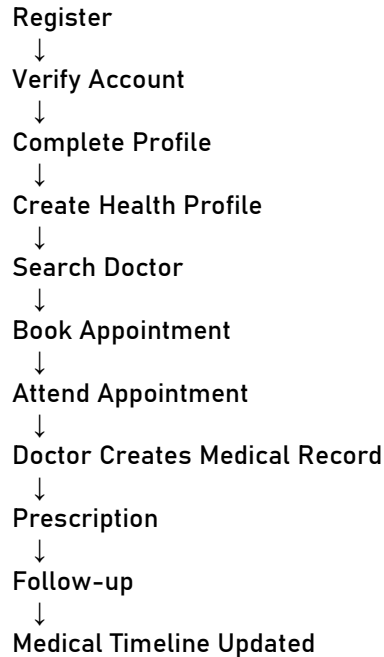
Strategy:

Primary Database
↓
Automated Backup
↓
Encrypted Backup Storage
↓

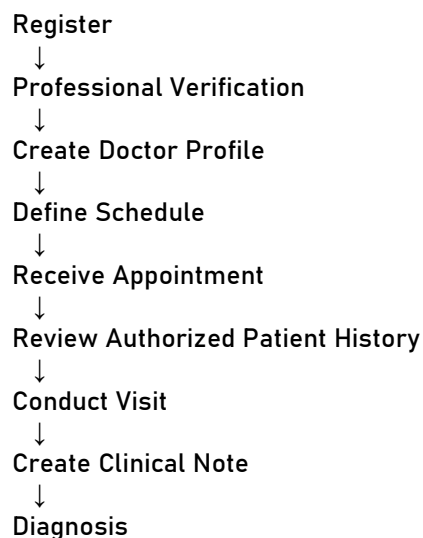
Recovery Testing

Backups should be tested, not merely created.

81. System Workflow — Patient



82. System Workflow — Doctor



↓
Prescription
↓
Follow-up

83. System Workflow — Medical Document

Patient Uploads Document
↓
File Validation
↓
Secure Storage
↓
Metadata Saved
↓
Optional OCR
↓
Structured Data
↓
Medical Timeline

84. System Workflow — Consent

Doctor Requests Access
↓
Patient Receives Notification
↓
Patient Reviews Scope
↓
Approve
↓
Access Granted
↓
Doctor Views Authorized Data
↓
Audit Log
↓
Consent Expires
↓
Access Revoked

85. Admin Dashboard

The administrator can manage:

Users

- Patients
- Doctors
- Clinics
- Organizations

Verification

- Doctor verification
- Clinic verification

Platform

- Appointments
- Reports
- Complaints
- Reviews

Security

- Audit logs
- Suspicious activity
- Account restrictions

Analytics

- Registered users
- Active doctors
- Appointments
- Platform usage

86. Analytics Dashboard

Business metrics:

Total Users

Active Users

Doctors

Patients

Appointments

Completed Appointments

Cancelled Appointments
Average Rating
Revenue

Healthcare metrics should be designed carefully to avoid exposing sensitive individual-level data unnecessarily.

87. Monetization Strategy

B2C

Free:

Medical profile
Medical history
Documents
Appointments
Basic reminders

Premium:

Advanced health analytics
Advanced reminders
AI summaries
Premium services

B2B

Doctors:

Monthly SaaS Subscription

Clinics:

Professional Plan

Hospitals:

Enterprise Plan

Laboratories:

Integration Fees

88. Revenue Streams

Potential revenue:

1. Doctor SaaS subscriptions
2. Clinic subscriptions
3. Enterprise hospital integrations
4. Paid follow-up consultations
5. Premium patient services
6. Pharmacy transaction fees
7. Laboratory integration
8. Insurance integrations

The platform should avoid business models that create incentives to misuse sensitive medical information.

89. MVP Scope

The first version should focus on the following.

Patient

- Registration
- Login
- Profile
- Medical profile
- Medical documents
- Medical timeline
- Doctor search
- Appointment booking
- Prescription viewing
- Medication reminders
- Consent management
- Notifications
- Doctor chat

Doctor

- Registration
- Verification
- Profile
- Schedule
- Appointments
- Patient history
- Medical notes
- Prescriptions
- Follow-up
- Chat

Admin

- User management
 - Doctor verification
 - Appointment monitoring
 - Basic analytics
 - Audit logs
-

90. Phase 2

After MVP:

- OCR
 - AI medical summary
 - AI pre-assessment
 - Lab integration
 - Radiology integration
 - Advanced analytics
 - Payment
 - Paid consultations
-

91. Phase 3

Enterprise ecosystem:

- Pharmacy
- Hospitals

- Insurance
 - FHIR integrations
 - Corporate healthcare
 - Advanced AI
 - Regional expansion
-

92. What Should NOT Be Built Initially

To avoid scope explosion, the graduation MVP should NOT initially attempt to fully implement:

- Complete Hospital Information System
- Complete Pharmacy ERP
- Complete Insurance System
- Full AI diagnosis system
- Full national healthcare infrastructure
- Full medical imaging platform
- Full electronic claims system

These should remain future roadmap items.

93. Six-Member Team Structure

Since the team consists of 6 members, a good structure is:

Member 1 — Team Lead / Backend Architect

Responsibilities:

- Architecture
 - Backend standards
 - Code review
 - Authentication
 - Integration
 - Technical decisions
-

Member 2 — Backend Developer

Responsibilities:

- Patient module
 - Medical records
 - Documents
 - Consent
-

Member 3 — Backend Developer

Responsibilities:

- Doctors
 - Appointments
 - Prescriptions
 - Follow-ups
-

Member 4 — Frontend Developer

Responsibilities:

- Patient application
 - Patient dashboard
 - Medical timeline
 - Appointments
-

Member 5 — Frontend Developer

Responsibilities:

- Doctor dashboard
 - Appointment management
 - Medical records
 - Prescriptions
-

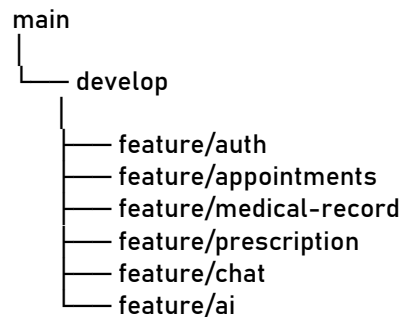
Member 6 — Full Stack / AI / Integration

Responsibilities:

- Chat
 - Notifications
 - AI services
 - Admin dashboard
 - Integrations
 - DevOps support
-

94. Git Workflow

Recommended:



Pull Requests should be reviewed before merging.

95. Coding Standards

Backend:

TypeScript
ESLint
Prettier
Clean Architecture principles
DTO validation
Centralized error handling

Frontend:

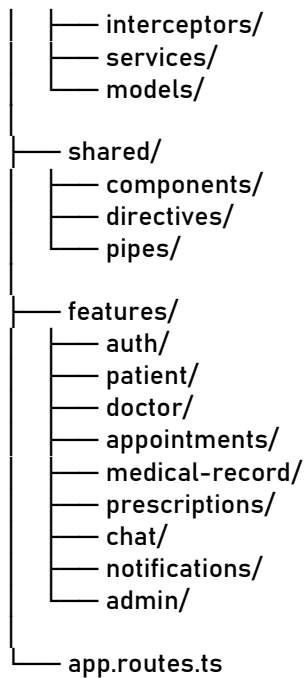
Angular
TypeScript
Reusable components
Reactive Forms
Guards
Interceptors
Lazy Loading
Feature-based structure

96. Backend Folder Structure

```
src/
├── config/
├── common/
│   ├── middleware/
│   ├── guards/
│   ├── errors/
│   ├── utils/
│   └── constants/
├── modules/
│   ├── auth/
│   ├── users/
│   ├── patients/
│   ├── doctors/
│   ├── appointments/
│   ├── medical-records/
│   ├── prescriptions/
│   ├── medications/
│   ├── documents/
│   ├── consent/
│   ├── chat/
│   ├── notifications/
│   ├── reviews/
│   ├── admin/
│   └── ai/
├── database/
├── app.ts
└── server.ts
```

97. Angular Folder Structure

```
src/app/
├── core/
└── guards/
```



98. Testing Strategy

Unit Testing

Test:

- Services
- Utilities
- Validators
- Business rules

Integration Testing

Test:

API

+

Database

+

Authentication

End-to-End Testing

Example:

Patient Login
↓
Search Doctor
↓
Book Appointment
↓
Doctor Login
↓
Confirm Appointment
↓
Create Prescription
↓
Patient Receives Prescription

99. Security Testing

Test against common web vulnerabilities:

- Broken access control
- Injection
- XSS
- CSRF where applicable
- Authentication weaknesses
- Rate-limit bypass
- IDOR/BOLA
- File upload vulnerabilities
- Token theft
- Privilege escalation

Healthcare applications require particular attention to authorization because a valid user account must not imply access to every patient's record.

100. Important Edge Cases

The system should handle:

Appointment

- Two users booking same slot
- Doctor cancelling
- Patient cancelling
- Doctor unavailable
- No-show
- Rescheduling

Medical Records

- Duplicate documents
- Invalid file
- Large file
- Deleted document
- Revoked access

Chat

- Expired consultation
- Blocked user
- Attachment failure
- Offline user
- Duplicate message

Consent

- Expired consent
- Revoked consent
- Partial permission
- Doctor requesting unauthorized data

101. Example Complete User Journey

A patient named Ahmed creates an account.

Register



Verify Phone



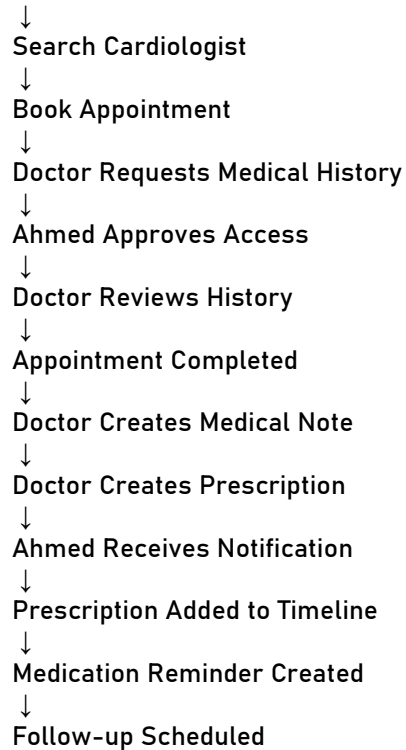
Create Medical Profile



Add Allergy



Upload Previous Lab Results



This single workflow demonstrates a significant portion of the platform.

102. Competitive Advantage

MediVault should not compete only as:

"Another doctor booking application."

Its primary differentiation is:

Healthcare Continuity

The patient history follows the patient.

Controlled Data Sharing

The patient controls access.

Structured Communication

Doctor communication is integrated with appointments and medical records.

Digital Healthcare Ecosystem

The platform can connect:

Patient
↓
Doctor
↓
Lab
↓
Radiology
↓
Pharmacy
↓
Hospital
↓
Insurance

103. Business Model

The platform can follow a hybrid model:

B2C
+
B2B
+
B2B2C

B2C

Patients.

B2B

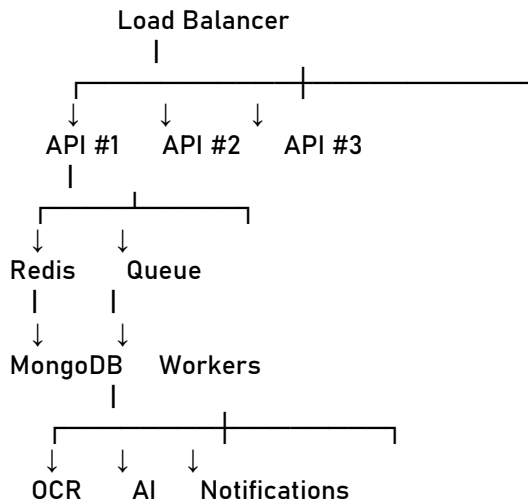
Doctors, clinics, laboratories, hospitals.

B2B2C

Insurance companies and corporate healthcare programs.

104. Future Scaling Architecture

When traffic increases:



Eventually modules can become independent services:

Auth Service
Appointment Service
Medical Record Service
Prescription Service
Chat Service
Notification Service
AI Service
Document Service

105. Microservices Migration Strategy

The system should not be rewritten completely.

Start:

Modular Monolith

Then extract high-load modules:

Chat
↓
Chat Microservice

Then:

Notification
↓
Notification Service

Then:

Document
↓
Document Service

Then:

AI
↓
AI Service

This provides an incremental scaling strategy.

106. Data Interoperability Strategy

The platform should internally use a domain model that can be mapped to FHIR.

Example:

Patient
→ FHIR Patient

Doctor
→ FHIR Practitioner

Visit
→ FHIR Encounter

Lab Result
→ FHIR Observation

Prescription
→ FHIR MedicationRequest

Medical Report
→ FHIR DiagnosticReport / DocumentReference

This does not mean the graduation project needs to implement the complete FHIR specification.

The goal is to make future interoperability possible.

107. DevOps

Recommended:

GitHub
Docker
CI/CD
Environment Variables
Automated Testing

Environments:

Development
Testing
Staging
Production

108. Docker Architecture

Possible containers:

Frontend
Backend
MongoDB
Redis
MinIO

Example:

```
docker-compose
├── frontend
├── backend
├── mongodb
├── redis
└── minio
```

109. Environment Variables

Sensitive values should never be committed to Git.

Example:

DATABASE_URL

JWT_SECRET
REFRESH_TOKEN_SECRET
REDIS_URL
STORAGE_ACCESS_KEY
STORAGE_SECRET_KEY
AI_API_KEY

Use:

.env

and proper secret management in production.

110. Project Success Metrics

Technical:

API Performance
Error Rate
System Availability
Test Coverage
Security Issues

Business:

Registered Doctors
Registered Patients
Appointments
Completed Appointments
Retention
Active Users

Product:

Booking Completion Rate
Medical Record Usage
Follow-up Usage
Doctor Adoption

111. Key Risks

Risk 1

Doctors may not adopt the platform.

Solution

Make doctor workflow extremely fast.

Risk 2

Patients may not upload historical records.

Solution

Provide:

- OCR
 - Easy upload
 - Timeline
 - Import tools
-

Risk 3

Security breach.

Solution

Security-first architecture, authorization, encryption, auditing, backups, and penetration testing.

Risk 4

Scope becomes too large.

Solution

Strict MVP.

Risk 5

AI produces incorrect information.

Solution

AI should assist rather than independently diagnose, with human review and clear safety boundaries.

112. MVP Definition

The project will be considered MVP-complete when:

Patient

- Can register.
- Can login.
- Can manage profile.
- Can manage medical history.
- Can upload documents.
- Can search doctors.
- Can book appointment.
- Can view prescriptions.
- Can receive notifications.
- Can communicate with doctor.
- Can manage access permissions.

Doctor

- Can register.
- Can be verified.
- Can manage profile.
- Can define availability.
- Can manage appointments.
- Can view authorized patient records.
- Can create medical notes.
- Can create prescriptions.
- Can communicate with patients.

Admin

- Can manage users.
 - Can verify doctors.
 - Can monitor platform.
 - Can view audit logs.
 - Can manage reports.
-

113. Future Roadmap

Version 1.0

Authentication
Patients
Doctors
Appointments
Medical Records
Prescriptions
Chat
Notifications
Consent
Admin

Version 2.0

OCR
AI Summary
AI Pre-Assessment
Payments
Lab Integration
Radiology Integration

Version 3.0

Pharmacy
Hospital Integration
Insurance
FHIR Integration
Enterprise SaaS

Version 4.0

Regional Healthcare Ecosystem
Cross-Provider Interoperability
Advanced Analytics
AI Clinical Assistance

114. Final System Vision

MediVault starts as:

Doctor Booking Platform

Then becomes:

Digital Medical Record

Then:

Healthcare Communication Platform

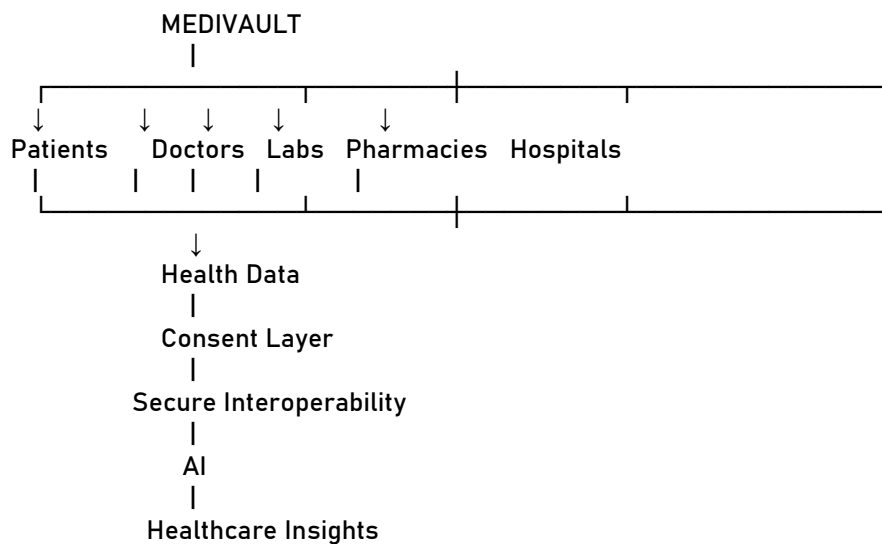
Then:

Healthcare Provider Network

And ultimately:

Digital Healthcare Ecosystem

The long-term ecosystem becomes:



115. Final Conclusion

MediVault is not intended to be another appointment-booking website.

The fundamental idea is to create a secure digital layer connecting the patient with the healthcare ecosystem.

The project addresses real problems:

- Fragmented medical history
- Paper-based medical records
- Unorganized doctor communication
- Difficult follow-up
- Lack of centralized medical information
- Limited patient control over medical data
- Lack of interoperability between healthcare providers

The initial graduation project should focus on building a strong MVP around:

Patient + Doctor + Medical Record + Appointment + Prescription + Consent + Chat

After establishing this foundation, the system can be expanded with:

AI + OCR + Labs + Radiology + Pharmacy + Hospitals + Insurance + FHIR

This gives the project both:

1. A realistic graduation-project scope.
2. A much larger commercial and technical vision.

The ultimate goal is to create a healthcare ecosystem in which a patient's medical journey becomes connected, organized, secure, portable, and controlled by appropriate consent, while doctors receive the information and tools they need to provide better continuity of care.

116. Project Tagline

"Your Health. Your History. Your Control."

Alternative:

"One Health Profile. Connected Care."

Alternative:

"Healthcare That Remembers."

117. Recommended Project Pitch

MediVault is a digital healthcare ecosystem that solves the problem of fragmented patient medical information and unorganized doctor-patient communication.

Instead of keeping medical history scattered between paper reports, different doctors, laboratories, and messaging applications, MediVault provides patients with a secure unified health profile where medical records, prescriptions, appointments, documents, and follow-ups can be organized in one timeline.

Patients control who can access their information through consent-based permissions, while doctors receive a structured view of the patient's authorized medical history.

The platform also provides digital prescriptions, follow-up communication, medication reminders, notifications, and future AI-assisted capabilities.

In its future roadmap, MediVault can connect laboratories, radiology centers, pharmacies, hospitals, and insurance companies through interoperable healthcare APIs.

Therefore, MediVault is not simply a doctor-booking application; it is designed as the foundation of a scalable digital healthcare ecosystem.

118. One-Line Pitch

MediVault is a secure digital healthcare ecosystem that gives patients one unified medical profile and enables authorized doctors and healthcare providers to access the right information at the right time.

119. Project Scope Philosophy

The project follows three principles:

1. Build Small

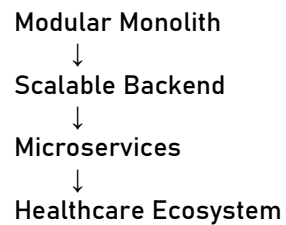
Start with a strong MVP.

2. Design Big

Architecture must support future expansion.

3. Scale Gradually

Move from:



This approach allows the graduation team to demonstrate a realistic working product while presenting a credible long-term technical and business vision.